

Measurable acceptance criteria — process scenarios

可度量的验收标准 — 流程场景

Each row carries all six required elements: scenario · preconditions · test data · actions · expected outcome · pass / fail.

每一行都包含全部六个必要要素：场景 · 前置条件 · 测试数据 · 操作步骤 · 预期结果 · 通过 / 失败判定。格内英文在上，中文在下。

#	Test scenario 测试场景	Preconditions 前置条件	Test data 测试数据	Actions 操作步骤	Expected outcome 预期结果	Pass / fail criteria 通过 / 失败判定
S1	Normal path — self-pay patient pays, is treated and discharged 正常路径 — 自费患者完成付款、接受治疗并出院	Camunda 8 running; Java workers started; no ledger row exists for the case Camunda 8 已运行；Java 工作器已启动；该病例在账本中尚无记录	case DEMO-JAVA-01 charge_amount "10.01" fundingStatus patient_payment_required 病例 DEMO-JAVA-01；金额 10.01；经费状态为患者需自付	Start process → register as referral → referral accepted → visit attended → P5 authorise treatment → letter → P7 patient payment required + 10.01 → P6 resources available → P5 authorise discharge 启动流程 → 按转诊登记 → 转诊被接受 → 到诊 → P5 授权治疗 → 发信 → P7 患者需付款且金额 10.01 → P6 资源可用 → P5 授权出院	Logs: request-payment SUCCESS ... publishing payment-result, then send-booking-confirmation SENT. Instance COMPLETED at end event Care episode ended. payment_ledger = 1 row, status SUCCESSFUL 日志：request-payment 为 SUCCESS，并发布 payment-result，随后 send-booking-confirmation 为 SENT。实例在结束事件 Care episode ended 处 COMPLETED。payment_ledger 恰好 1 行，状态 SUCCESSFUL	PASS = all four observations. FAIL = any log missing, or the instance is still ACTIVE. 通过 = 上述四项观察全部成立。失败 = 任一日志缺失，或实例仍为 ACTIVE。
S2	Exception — payment is unsuccessful (demo rule: amount ends in 0) 异常路径 — 付款未成功（演示规则：金额以 0 结尾）	Same as S1 与 S1 相同	case DEMO-JAVA-01 charge_amount "10.00" 病例 DEMO-JAVA-01；金额 10.00	Same as S1 up to and including P7 与 S1 相同，做到 P7（含）为止	Log request-payment UNSUCCESSFUL. mark-payment-investigate writes payment_ledger status INVESTIGATE. Token waits at the human task Resolve funding issue. No second charge is issued 日志 request-payment 为 UNSUCCESSFUL。mark-payment-investigate 把 payment_ledger 写成 INVESTIGATE。令牌停在人工任务 Resolve funding issue。不会发起第二次扣款	PASS = token is NOT at BookTreatment and the ledger holds exactly 1 INVESTIGATE row. FAIL = booking reached, or more than 1 ledger row. 通过 = 令牌不在 BookTreatment，且账本恰好 1 行 INVESTIGATE。失败 = 已走到预约，或账本超过 1 行。
S3	Alternative — funding already settled, provider is not called 替代路径 — 经费已经落实，不调用支付提供方	Case has reached P7 病例已到达 P7	fundingStatus "hospital_funded" (also insurer_approved / patient_paid / arrangement) charge_amount empty 经费状态为 hospital_funded（也可为 insurer_approved / patient_paid / arrangement）；金额为空	Complete P7 with that funding outcome 用该经费结果完成 P7	No request-payment log line at all. Token goes straight to P6 Confirm treatment booking. payment_ledger has 0 rows for the case 完全没有 request-payment 日志。令牌直接进入 P6 Confirm treatment booking。该病例的 payment_ledger 为 0 行	PASS = zero payment logs and zero ledger rows. FAIL = any provider call recorded. 通过 = 支付日志为零，且账本行为零。失败 = 记录到任何一次提供方调用。
S4	Exception — a clinical change that moves money, refund recorded 异常路径 — 涉及金额变动的临床变更，并记录退款	An active case has reached the clinical-change step (P11/P12) 一个进行中的病例已到达临床变更步骤（P11/P12）	case DEMO-001 financeAdjustment "resolved" charge_amount "25.50" 病例 DEMO-001；财务调整为 resolved；金额 25.50	P11/P12 authorise modification → moneyAffected = true → P7/P11/P12 approve and record refund P11/P12 授权修改 → moneyAffected = true → P7/P11/P12 批准并记录退款	request-refund returns refund_status SUCCESSFUL, refund_recorded true, refund_transaction_reference starting RF-. Ledger amount 25.50. audit_event action request-refund. Process continues to the revised clinical plan request-refund 返回：refund_status 为 SUCCESSFUL，refund_recorded 为 true，退款参考号以 RF- 开头。账本金额 25.50。audit_event 的 action 为 request-refund。流程继续进入修订后的临床计划	PASS = all four returned values match and there is exactly 1 ledger row. FAIL = missing audit row, or status ≠ SUCCESSFUL. 通过 = 四项返回值全部相符，且账本恰好 1 行。失败 = 缺少审计行，或状态不是 SUCCESSFUL。
S5	Alternative — urgent care requires clinician authorisation 替代路径 — 紧急治疗需要临床医生授权	Case has reached P7 病例已到达 P7	fundingStatus "urgent_review" then urgentDecision authorise hold 经费状态为 urgent_review；随后 urgentDecision 取 authorise 或 hold	Complete P7 with urgent_review → P8 clinician authorises urgent care with reason 用 urgent_review 完成 P7 → P8 临床医生附理由授权紧急治疗	authorise → urgentCase true and the token reaches P6 booking. hold → the token returns to P7 Funding and never reaches booking authorise → urgentCase 为 true，令牌到达 P6 预约。hold → 令牌回到 P7 Funding，不会到达预约	PASS = routing matches the decision. FAIL = booking is reached after hold. 通过 = 路由与决定一致。失败 = hold 之后仍然到达预约。

S1–S3 are demonstrated live in Camunda Tasklist; S4–S5 are demonstrated by the console log and the Operate variable view.

S1–S3 在 Camunda Tasklist 里现场演示；S4–S5 用控制台日志和 Operate 的变量视图演示。

Measurable acceptance criteria — worker-level and negative scenarios

可度量的验收标准 — 工作器级别与反向场景

These are the checks that prove the external workers are safe to retry and safe to fail.

这些检查用来证明外部工作器在重试时是安全的，在失败时也是安全的。格内英文在上，中文在下。

#	Test scenario 测试场景	Preconditions 前置条件	Test data 测试数据	Actions 操作步骤	Expected outcome 预期结果	Pass / fail criteria 通过 / 失败判定
S6	Worker idempotency — the same payment is requested twice 工作器幂等 — 同一笔付款被请求两次	S1 has already completed once for this case 该病例的 S1 已经完成过一次	case DEMO-JAVA-01 payment_requested_once = true 病例 DEMO-JAVA-01; payment_requested_once = true	Trigger request-payment again with the same case reference 用同一病例号再次触发 request-payment	Log: request-payment idempotent reuse. Returned payment_status and transaction_reference are unchanged. payment_ledger row count stays 1 日志: request-payment idempotent reuse。返回的 payment_status 与 transaction_reference 不变。payment_ledger 行数仍为 1	PASS = identical values and 1 row. FAIL = a new row, or a new transaction reference. 通过 = 返回值相同且只有 1 行。失败 = 出现新行，或出现新的交易参考号。
S7	Worker idempotency — the same refund is requested twice 工作器幂等 — 同一笔退款被请求两次	Clean ledger and audit table 账本表与审计表均为空	case A-REF-003 financeAdjustment "resolved" charge_amount "12.00" 病例 A-REF-003; 财务调整为 resolved; 金额 12.00	Call request-refund twice with exactly the same variables 用完全相同的变量调用 request-refund 两次	The second call returns exactly the same map as the first. Ledger count = 1. Audit count = 1 第二次调用返回的映射与第一次完全相同。账本行数 = 1。审计行数 = 1	PASS = both counts are exactly 1. FAIL = any duplicate row in either table. 通过 = 两个计数都恰好为 1。失败 = 任一表中出现重复行。
S8	Negative — invalid payment input must write nothing 反向测试 — 无效的付款输入不得写入任何数据	Clean ledger and audit table 账本表与审计表均为空	variables with no case_reference and no patientId 变量中没有 case_reference，也没有 patientId	Call request-payment 调用 request-payment	The job fails with error code payment-input-invalid. No message is published. payment_ledger count = 0. audit_event count = 0 作业以错误码 payment-input-invalid 失败。不发布任何消息。payment_ledger 行数 = 0。audit_event 行数 = 0	PASS = both counts are 0. FAIL = any row written despite the failure. 通过 = 两个计数都为 0。失败 = 失败之后仍写入了任何一行。
S9	Negative — a money-affected change is never auto-refunded 反向测试 — 涉及金额的变更绝不会自动退款	Clean audit table 审计表为空	changeTarget "discharge" moneyAffected = true changeTarget 为 discharge; moneyAffected = true	Call notify-care-change 调用 notify-care-change	careChangeNotificationStatus = SENT_FINANCE_REVIEW_REQUIRED. The result contains no financeAdjustment or fundingStatus key. The token still waits at Finance careChangeNotificationStatus = SENT_FINANCE_REVIEW_REQUIRED。结果中不含 financeAdjustment 或 fundingStatus。令牌仍停在 Finance	PASS = status is exactly SENT_FINANCE_REVIEW_REQUIRED. FAIL = the worker issues a refund by itself. 通过 = 状态恰好是 SENT_FINANCE_REVIEW_REQUIRED。失败 = 工作器自行发起了退款。
S10	Deployability — model and code cannot drift apart 可部署性 — 模型与代码不能各走各的	Repository checked out; Python 3 available 仓库已检出; 本机有 Python 3	Coursework/hospital-pathway (all sources) Coursework/hospital-pathway (全部源码)	Run: python scripts/test_member_d_bpmn.py 运行: python scripts/test_member_d_bpmn.py	Every zeebe:taskDefinition type in the BPMN has a matching @JobWorker(type=...) in Java, and every sequence-flow reference agrees BPMN 中每一个 zeebe:taskDefinition 的 type，在 Java 里都有对应的 @JobWorker(type=...)，且每一条顺序流引用都一致	PASS = the unittest run reports OK. FAIL = any job type without a worker. 通过 = 单元测试运行结果为 OK。失败 = 存在没有工作器的作业类型。

S6–S9 are automated JUnit tests (MemberAPathwayWorkersTest, MemberDPathwayWorkersTest) against a real H2 database; S10 is a Python test over the BPMN XML.

S6–S9 是对着真实 H2 数据库跑的自动化 JUnit 测试 (MemberAPathwayWorkersTest、MemberDPathwayWorkersTest) ；S10 是针对 BPMN XML 的 Python 测试。